PROGRAMOWANIE OBIEKTOWE W C++

Część 2: Klasy, Enkapsulacja i Konstruktory

Prowadzący: dr Marcin Misiaszek

Rok akademicki 2025/2026

1. Dlaczego Programowanie Obiektowe (OOP)?

Programowanie proceduralne (C) skupia się na **funkcjach** operujących na danych. OOP skupia się na **obiektach**, które łączą dane i zachowania.

Trzy filary OOP:

- **Enkapsulacja (Hermetyzacja)** – Ukrywanie stanu wewnętrznego i udostępnianie interfejsu.
- **Dziedziczenie** – Tworzenie nowych klas na bazie istniejących.
- **Polimorfizm** – Jednolity interfejs dla różnych typów danych.

> **Obiekt = Stan (Dane) + Zachowanie (Metody)**

2. Od Struktur do Klas

W C++

```
struct
```

i

```
class
```

są niemal identyczne. Różnią się jedynie **domyślnym poziomem dostępu**.

Struktura (struct)

- Domyślnie: public
- Używana głównie dla prostych paczek danych (POD - Plain Old Data).

Klasa (class)

- Domyślnie: private
- Używana, gdy chcemy chronić dane i wymusić interfejs.

```
class Date {
    int m_year; // Prywatne domyślnie!
public:
    void print(); // Publiczne
};
```

3. Specyfikatory Dostępu: public vs private

Kontrola nad tym, kto może modyfikować wnętrze naszego obiektu.

- **private:** Dostępne tylko dla metod wewnątrz tej samej klasy. Chroni przed błędami logicznymi z zewnątrz.
- **public:** Interfejs klasy. Widoczny dla każdego, kto posiada obiekt.

> Złota zasada: Pola danych (zmienne członkowskie) powinny być **zawsze prywatne**.

```
private:
    int m_month { 1 }; // m_ oznacza "member" (pole klasy)
```

4. Gettery i Settery (Akcesory i Mutatory)

Skoro dane są prywatne, jak z nich korzystać? Przez publiczne metody.

Zalety stosowania metod zamiast bezpośredniego dostępu:

- **Walidacja:** Setter może sprawdzić, czy miesiąc nie wynosi 13.
- **Tylko do odczytu:** Możemy udostępnić tylko getter (brak settera = stała dla zewnątrz).
- **Abstrakcja:** Możemy zmienić sposób przechowywania danych bez zmiany kodu użytkownika.

```
int getMonth() const { return m_month; } // Getter
void setMonth(int m) {
    if (m >= 1 && m <= 12) m_month = m; // Walidacja!
}
```

5. Metody Pomocnicze i Spójność Stanu

Nie każda metoda musi być publiczna. Metody prywatne pomagają organizować kod wewnątrz klasy.

Utrzymywanie inwariantów (spójności):

Obiekt nie powinien nigdy wejść w stan nielogiczny.

```
class Employee {
    std::string m_name;
    char m_initial;

public:
    void setName(std::string_view name) {
        m_name = name;
        updateInitial(); // Metoda pomocnicza
    }
private:
    void updateInitial() { m_initial = m_name[0]; }
};
```

6. Konstruktory: Narodziny Obiektu

Konstruktor to specjalna metoda wywoływana automatycznie w momencie tworzenia obiektu.

- Ma taką samą nazwę jak klasa.
- **Nie posiada** typu zwracanego (nawet void).
- Może być przeciążany (wiele konstruktorów dla jednej klasy).

```
class Foo {
public:
    Foo() { /* domyślny */ }
    Foo(int x, int y) { /* parametryczny */ }
};

Foo f1;        // Woła domyślny
Foo f2{ 5, 10 }; // Woła parametryczny
```

7. Lista Inicjalizacyjna Członków

Najważniejszy mechanizm inicjalizacji w C++. Wykonywany **zanim** wejdziemy do klamerek konstruktora.

> **Wydajność: Zapobiega domyślnej inicjalizacji pola, a następnie nadpisywaniu go wewnątrz konstruktora.**

```
// NIEPOLECANE (Przypisanie):
Foo(int x) { m_x = x; }

// POLECANE (Inicjalizacja):
Foo(int x) : m_x{ x } // Lista inicjalizacyjna
{
}
```

Wymagane dla: pól

```
const
```

, referencji oraz klas bez konstruktora domyślnego.

8. Dobre Praktyki (Best Practices)

- **Const Correctness:** Każda metoda, która tylko czyta dane, powinna mieć

  ```
  const
  ```

  na końcu:

  ```
  int getValue() const;
  ```

  .

- **Nazewnictwo:** Używaj

  ```
  m_
  ```

  dla pól prywatnych. To natychmiast odróżnia je od zmiennych lokalnych i parametrów.

- **Single Responsibility:** Klasa powinna odpowiadać za jedną rzecz.
- **std::string_view:** Używaj jako parametrów w setterach dla lepszej wydajności (brak kopiowania).

ANALIZA PROGRAMÓW ĆWICZENIOWYCH

Ewolucja kodu: Od struktury do klasy

Program cw2-1.cpp: Podejście proceduralne

Punkt wyjścia: klasyczna struktura w stylu C, używana jako zwykły kontener na dane (Plain Old Data).

Zastosowane techniki:

- **Agregat danych:** Definicja

  ```
  struct Date
  ```

  z polami inicjalizowanymi wartością zero za pomocą

  ```
  {}
  ```

  .

- **Referencja do const:** Funkcja

  ```
  print(const Date& date)
  ```

  – wydajne przekazywanie bez kopiowania i gwarancja niezmienności obiektu.

- **Inicjalizacja agregatu:** Składnia

  ```
  Date today { 2020, 10, 14 }
  ```

  przypisująca wartości w kolejności deklaracji.

Program cw2-2.cpp: Pierwsza metoda

Pierwszy krok w stronę obiektowości – połączenie danych i zachowania wewnątrz jednego typu.

Zastosowane techniki:

- **Metoda członkowska:** Definicja

  ```
  print()
  ```

  wewnątrz struktury. Funkcja staje się częścią typu.
- **Niejawny dostęp:** Metoda ma bezpośredni dostęp do pól obiektu bez potrzeby przekazywania ich jako argumentów.
- **Operator kropki:** Jednolita składnia dla pól i metod:

  ```
  today.print()
  ```

  .

> **Wniosek: Obiekt staje się samowystarczalny. Typ danych zaczyna definiować nie tylko "co to jest", ale i "co to potrafi zrobić".**

Program cw2-3.cpp: Enkapsulacja i Klasy

Wprowadzenie ścisłej kontroli dostępu (hiding) i świadomego projektowania interfejsu.

Zastosowane techniki:

- **class vs struct:** Zmiana słowa kluczowego wymusza domyślną prywatność (

  ```
  private
  ```

).
- **In-class member initializers:** Ustawienie bezpiecznych wartości domyślnych przy deklaracji pól (C++11).
- **Specyfikatory dostępu:** Jawny podział na sekcję publiczną (interfejs) i prywatną (implementacja).
- **Metoda const:**

  ```
  void print() const
  ```

  – deklaracja, że metoda nie modyfikuje stanu obiektu (tylko odczyt).

Program cw2-4.cpp: Kontrola dostępu (1/2)

Pełny model klasy wykorzystujący akcesory i mutatory do zarządzania stanem.

Zastosowane techniki:

- **Gettery:** Metody

  ```
  const
  ```

  zwracające wartości prywatnych pól bez ich kopiowania (jeśli typ na to pozwala).
- **Settery:** Metody umożliwiające modyfikację prywatnych pól przez publiczny interfejs.
- **Zalety abstrakcji:** Możliwość zmiany wewnętrznej reprezentacji danych (np. zmiana typu pola) bez konieczności modyfikacji kodu w

  ```
  main
  ```

  .

Program cw2-4.cpp: Kontrola dostępu (2/2)

Dalsza analiza:

- **Punkt walidacji:** Settery stanowią idealne miejsce do dodania logiki sprawdzającej poprawność danych (np. zakres miesiąca 1-12).
- **Hermetyzacja logiczna:** Możemy kontrolować nie tylko to, KTO zmienia dane, ale JAKIE wartości są dozwolone.

> **Wniosek: Gettery i settery tworzą bezpieczny most między prywatnym wnętrzem klasy a światem zewnętrznym.**

Program cw2-5.cpp: Spójność wewnętrzna (1/2)

Przykład klasy dbającej o tzw. inwarianty – logiczne powiązania między polami.

Zastosowane techniki:

- **Zależności danych:** Metoda

  ```
  setName()
  ```

  aktualizuje jednocześnie dwa pola (

  ```
  m_name
  ```

  i

  ```
  m_firstInitial
  ```

  ), dbając o ich spójność.

- **std::string_view:** Użycie nowoczesnego typu (C++17) zapewniającego wydajny "widok" na tekst bez kosztownego kopiowania

  ```
  std::string
  ```

  .

Program cw2-5.cpp: Spójność wewnętrzna (2/2)

Dalsza analiza:

- **Zapewnienie inwariantu:** Klasa uniemożliwia powstanie stanu niespójnego (np. imię "Mark", a inicjał 'J').
- **name.front():** Wykorzystanie metod standardowych typów do pobierania składowych danych.

> **Wniosek: Enkapsulacja pozwala klasie pilnować własnych reguł logicznych, chroniąc program przed błędami stanu.**

Program cw2-6.cpp: Cykl życia - Konstruktor (1/2)

Zrozumienie mechanizmu automatycznego powoływania obiektów do życia.

Zastosowane techniki:

- **Definicja konstruktora:** Specjalna metoda o nazwie klasy, bez typu zwracanego, wywoływana automatycznie.
- **Konstruktor parametryczny:** Możliwość przekazywania wartości początkowych podczas tworzenia obiektu:

  ```
  Foo foo{ 6, 7 }
  ```

  .

Program cw2-6.cpp: Cykl życia - Konstruktor (2/2)

Dalsza analiza:

- **Pułapka inicjalizacji:** Przykład pokazuje, że samo otrzymanie parametrów przez konstruktor NIE oznacza ich automatycznego zapisania do pól klasy.
- **Inicjalizacja domyślna:** Jeśli konstruktor nie wykona przypisania, pola zachowują wartości z in-class initializers (0, 0).

> **Wniosek: Konstruktor to miejsce na konfigurację obiektu, ale programista musi jawnie zdecydować, co zrobić z przekazanymi danymi.**

Program cw2-7.cpp: Inicjalizacja (1/2)

Najbardziej efektywny i idiomatyczny sposób inicjalizacji członków klasy w nowoczesnym C++.

Zastosowane techniki:

- **Member Initializer List:** Składnia

  ```
  : m_x{x}, m_y{y}
  ```

  przed ciałem konstruktora.
- **Wydajność:** Inicjalizacja odbywa się bezpośrednio, bez tworzenia obiektów domyślnych i ich późniejszego nadpisywania.

Program cw2-7.cpp: Inicjalizacja (2/2)

Dalsza analiza:

- **Wymogi techniczne:** Niektóre pola (typy

  ```
  const
  ```

  , referencje) MUSZĄ być inicjalizowane na liście.
- **Czystość kodu:** Ciało konstruktora często pozostaje puste, służąc jedynie do ewentualnych logów lub dodatkowej logiki.

> **Wniosek: Lista inicjalizacyjna to standard w C++. Zapewnia poprawność, wydajność i czytelność kodu.**

9. Podsumowanie Wykładu

1. **Klasa** to połączenie enkapsulacji danych (private) i metod operujących na nich (public).
2. Bezpośredni dostęp do danych jest zabroniony – komunikacja odbywa się przez **Gettery i Settery**.
3. Metody typu **const** gwarantują bezpieczeństwo – dają pewność, że stan obiektu nie zostanie zmieniony podczas odczytu.
4. **Konstruktor** automatyzuje inicjalizację, a **Lista Inicjalizacyjna** jest najbardziej wydajnym sposobem jego implementacji.

Zadania do wykonania na ćwiczeniach

- Zaimplementuj klasę

  ```
  Rectangle
  ```

  z prywatnymi polami

  ```
  m_width
  ```

  i

  ```
  m_height
  ```

  .

- Stwórz konstruktor wykorzystujący **listę inicjalizacyjną**.
- Dodaj settery z **walidacją** (np. szerokość musi być większa od zera).
- Dodaj metodę pomocniczą

  ```
  double calculateArea() const
  ```

  .

- W

  ```
  main()
  ```

  przetestuj działanie walidacji, próbując ustawić ujemną szerokość.